Noesis: Selective Memory Injection for Cross-Tool AI Conversation Consistency

Peizheng Du
Independent Researcher
peterdu0628@gmail.com

October 2026

Abstract

Users converse with several AI assistants, yet each tool's memory is isolated. Shared and evolving memory layers already exist; this paper studies a narrower design choice: an explicit rule that decides whether a captured statement is firm enough to enter the prompt. **Noesis** is a local-first memory layer that extracts typed *thoughts* from conversations across tools into a shared per-user store, injects a thought only after a confidence lifecycle (repetition, assertion strength, cross-tool corroboration, no contradiction) promotes it from *tentative*, and caps injected context with a token budget. On 110 paired synthetic personal questions, memory injection raises the keyword-hit rate of Gemini Flash-Lite, gemma3:4b, and qwen2.5:3b by 43.6, 48.2, and 25.5 percentage points (all $p < 0.001$, McNemar's test). At 500 stored memories Noesis injects about 207 tokens, 35 times fewer than injecting everything, with 9 of 10 targets at rank 1; a standard top-5 RAG ranks as well with fewer tokens, so Noesis's distinct gain is filtering: in two constructed tests, ungated retrieval surfaced tentative or hedged statements 19 and 13 times, and Noesis surfaced none. Its gate judges how firmly something was said, not whether it is true. On strictly cross-tool queries Noesis retrieves 37% of targets, where per-tool isolation is bound to 0%. The code, experiment harness, and raw results are released.

Keywords: large language models; LLM memory; retrieval-augmented generation; personalization; cross-tool memory; confidence gating; local-first software

1 Introduction

Heavy users of AI assistants switch between ChatGPT, Claude, Gemini, and local models, and each tool keeps its own memory: a preference explained to Claude must be re-explained to GPT, and the positions and open questions a user works out in one tool are lost at every tool boundary.

Two families of systems address parts of this problem. *Memory layers* such as Mem0 [4] extract and consolidate facts; OpenMemory MCP [18] shares one local store across MCP clients; Zep [22] and Hindsight [13] track how facts change and how firmly opinions are held (§2). *Retrieval-augmented generation* (RAG) [14] retrieves context per query, but naive variants inject everything retrieved, so cost grows with memory size and the relevant signal is buried (I measure 7,192 tokens at n=500).

Noesis is built on a different premise: *what* gets injected matters as much as *whether* anything is injected. Thoughts should earn their way into the context the way beliefs earn their way into a mind: through repetition, consistency, and survival against contradiction. Earning here means passing an admission rule, not being verified as true.

This paper makes the following contributions:

1. **A confidence lifecycle for conversational memory** (§3). Thoughts of seven types mature tentative→provisional→settled via a four-signal scorer, with type-specific decay and supersession that cut stale-stance leakage from 93% to 7% (§3.3).

2. **Gated injection that scales** (§5.2–§5.3). At 500 memories Noesis injects ∼207 tokens (35 times fewer than injecting everything) at MRR 0.90; top-5 RAG matches that ranking more cheaply, but only Noesis's gate keeps tentative and hedged statements out (0 vs. 19 and 0 vs. 13 in constructed tests).

3. **Quantified cross-tool memory sharing** (§5.4). A shared per-user store retrieves 37% of strictly cross-tool targets, where per-tool isolation scores 0% by construction (50% vs. 7.5% over all 40 scenarios).

4. **An A/B result across three model tiers** (§5.1). On identical n=110 question sets, memory injection significantly raises the keyword-hit rate for a cloud model (Gemini Flash-Lite, +43.6pp), a mid-tier local model (gemma3:4b, +48.2pp), and a small local model (qwen2.5:3b, +25.5pp); all $p < 0.001$.

The system is local-first: embeddings and retrieval run on-device (sqlite-vec + all-MiniLM-L6-v2), and conversation text leaves the machine only for optional cloud extraction or when the proxy forwards a request, with its injected memories, to a cloud model. Code, experiment harness, saved raw results, and a script that recomputes every reported number are released.[1]

2 Related Work

Sources were checked on 17 October 2026; the references mark preprints and vendor announcements, and `docs/related_systems.md` tabulates what each cited version describes.

Memory for LLM agents. MemGPT [20] pages memory between context tiers, and Letta adds archival storage. Mem0 [4] extracts and consolidates facts with an LLM pipeline and has a graph-based variant; A-MEM [25] links memories into an evolving note network; Zep [22] stores them in a temporal knowledge graph and invalidates contradicted facts with an LLM, more generally than Noesis's supersession rules (§3.3), but describes no threshold that keeps a valid fact out. Generative Agents [21] rank memories by recency, importance, and relevance, and MemoryBank [28] lets them fade along a forgetting curve, the closest precedents for Noesis's type-specific decay. OpenMemory MCP [18] already shares one local store across MCP clients. Noesis differs in (a) an explicit admission rule, under which nothing below provisional is injected and which the OpenMemory announcement does not describe, and (b) entry points beyond MCP (an OpenAI-compatible proxy and a browser extension). Its candidate contribution is therefore the admission policy, not memory beyond a context window in general. Benchmarks such as LoCoMo [16], LongMemEval [23], and the in-progress LongMemEval-V2 [24] are not yet used here.

Hindsight. Hindsight [13], the closest system, separates world facts, the agent's experiences, opinions, and entity observations, recalls through four channels under a token budget, and gives each opinion an evolving confidence, so typed memory, budgeted multi-channel retrieval, and evolving confidence are not Noesis contributions. Its confidence belongs to opinions the *agent* forms and, in the version I examined, does not gate recall, whereas Noesis gates the *user's* own statements, runs

[1] https://github.com/AngryKiwiWithLegs/Noesis

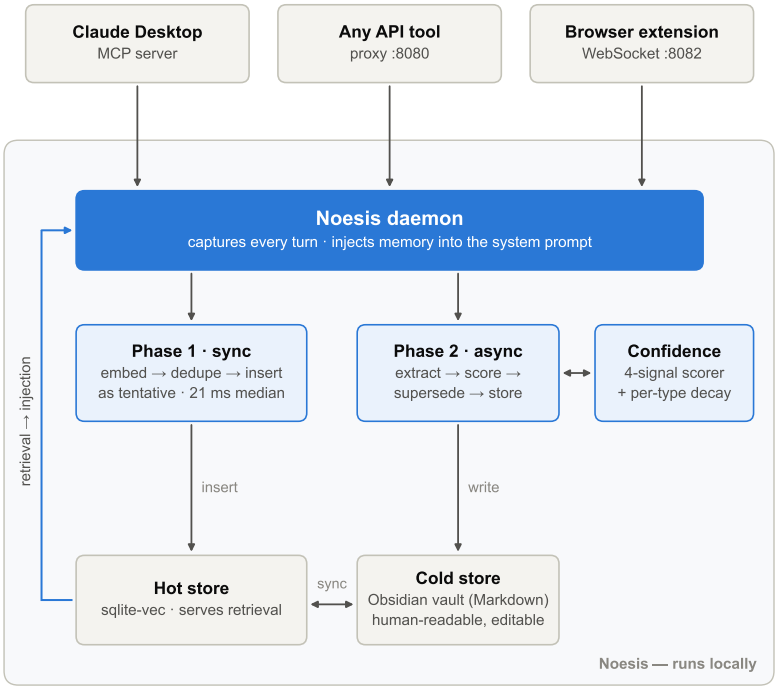


Figure 1: Noesis architecture. Tools connect through an MCP server, an OpenAI-compatible proxy, or a WebSocket; Phase 1 stores each utterance at once and Phase 2 extracts, scores, and links it in the background. A hot store (sqlite-vec) serves retrieval and a cold store (Obsidian vault) keeps memory editable.

on SQLite with a CPU embedding model, and pools several AI tools into one store. I have not compared the two empirically.

Learned and graph-structured memory control. Surveys organize agent memory by forms, functions, and dynamics [10] and graph memory by extraction, storage, retrieval, and evolution [26]. GraphRAG [6] summarizes entity communities across a corpus, a different task from tracking one user's preferences. AgeMem [27] learns when to store, retrieve, update, summarize, or discard memories, and MRAgent [11] explores an associative memory graph guided by reasoning. Noesis uses fixed rules instead; its intended advantages (transparency, low overhead, portability) are not measured here beyond the latencies in §3.1.

Memory security. MINJA [5] plants malicious memories through ordinary queries, and a survey [15] argues that memory security spans writing, storage, retrieval, execution, sharing, and forgetting and cannot be added at retrieval alone. Noesis's gate acts at retrieval and judges how firmly something was said, so a firmly worded planted statement passes (§9), and a shared store spreads it to every tool.

Retrieval-augmented generation. RAG [14] and its successors improve per-query retrieval for corpus question answering, not a user's evolving personal memory. Noesis pairs hybrid retrieval with a hard token budget and a confidence gate; §5.3 shows why injecting everything fails for personal memory.

Personalization. Chat vendors' memory features (e.g., ChatGPT memory) are per-product and closed; Noesis scopes memory to the user rather than the tool (§5.4).

Visualization of embedding spaces. Force-directed layouts [7] place points by attraction and repulsion; the star map (§6) adds weak time and confidence springs to a 3D force-directed layout of thought embeddings.

3 System Design

Noesis sits between AI tools and model providers (Figure 1). API clients point at a local OpenAI-compatible proxy, which injects memory into the system prompt, forwards the request to the configured provider (cloud or local), and captures the conversation for memory formation; Claude Desktop connects through an MCP server and the browser extension through a WebSocket.

3.1 Thought taxonomy and two-phase capture

Conversational memory is typed into seven categories with distinct persistence semantics (Table 1). An *identity* thought ("I am a backend engineer") should outlive an *event* ("I tried Bun yesterday"), and the taxonomy encodes this as type-specific half-lives.

Capture is two-phase. *Phase 1* is synchronous: embed, deduplicate by hash, insert as *tentative* (median 21 ms, p99 684 ms over 100 adds on an x86-64 Mac). *Phase 2* runs in the background: an extractor (a small cloud LLM, or a deterministic fallback) classifies the thought, a confidence scorer updates its status, a supersession check retires contradicted predecessors, and the cold store (Markdown in an Obsidian vault) is updated for people to read and edit. Building the injected context adds a median of 21 ms (p99 55 ms) per request.

Table 1: Thought taxonomy with type-specific half-lives and injection priority.

Type	**Definition**	**Half-life (days)**	**Priority**
identity	Who the user is	∞	highest (always)
preference	Explicit preference	90	high
position	Stance on a topic	90	high
question	Open question under exploration	30	medium
synthesis	Insight from merged ideas	180	medium
event	Happened / decided	7	low (recent)
contradiction	Self-contradictory judgment	∞	flagged

3.2 Confidence lifecycle

Every thought begins *tentative*: stored, but never injected and invisible to retrieval—the "capture threshold low, injection threshold high" principle. Four signals promote a thought: *repetition* (the same stance recurs across sessions), *assertion strength* ("I've decided" outranks "maybe"), *cross-tool consistency* (distinct phrasings of a stance from different tools), and *absence of contradiction*. The scorer combines them as $0.35\times$repetition$+0.25\times$assertion$+0.25\times$no-contradiction$+0.15\times$cross-tool, each scored from 0 to 1; a thought becomes provisional at 0.45 and settled at 0.75. A single firmly stated thought with no conflict ("I prefer X", assertion 0.8) therefore scores exactly 0.45 and becomes injectable, while settling always requires repetition: at least three similar earlier thoughts

with cross-tool corroboration, or six without. *Provisional* thoughts inject when topic-relevant; *settled* thoughts always inject. The result is a rule-based admission score: it decides what may be injected, not whether a thought is true, correctly attributed, still valid, or cleared for reuse, and a score of 0.75 is not a calibrated 75% probability of being correct [9]. Confidence evolves incrementally: reinforcement $+0.10\,(1 - C)$, conflict -0.20, and time decay $C \leftarrow C\,e^{-0.693\,\Delta t/t_{1/2}}$ with $t_{1/2}$ set by type (Table 1).

3.3 Cluster-aware supersession

When a user changes their mind ("I moved from MySQL to PostgreSQL"), the old stance should retire, but embedding similarity is the wrong signal: contradictory stances mention different entities. Noesis requires jointly (1) a replacement verb ("switched to", "migrated to", …, EN+ZH), (2) the same domain via a keyword map, (3) stance-like type, and (4) injectable status of the new thought. Matched predecessors are marked *superseded*, excluded by every retrieval path, and linked via `superseded_by`. On the same 30 scenarios, run before and after the fix, this reduced stale-stance leakage from 93% to 7% (Table 2).

Table 2: Supersession fix: stale-stance leakage before/after (n=30 scenarios).

Metric	**Before**	**After**	**Δ**
Old stance leaked into context	28/30 (93%)	2/30 (7%)	−87pp
New stance surfaced	12/30 (40%)	28/30 (93%)	+53pp
Ideal (new shown, old retired)	1/30 (3%)	27/30 (90%)	+87pp

3.4 Selective retrieval and injection

At query time (Figure 2), the semantic channel retrieves up to eight thoughts by dense similarity; once a user has 300 thoughts it also runs BM25 and fuses the two rankings by reciprocal-rank fusion with time-decay weighting. Up to three recent and three core (identity and preference) thoughts are then appended, duplicates are dropped, and a confidence gate excludes everything below *provisional*. A hard token budget (1,200 tokens) truncates the remainder, and the result is injected into the system prompt as compact structured lines. A one-hop expansion over vault links currently adds nothing (§7).

4 Evaluation Setup

I evaluate four research questions:

- **RQ1**: Does memory injection raise the keyword-hit rate on personal questions? (A/B, end-to-end, §5.1)
- **RQ2**: Which components contribute? (ablation, retrieval-layer, §5.2)
- **RQ3**: How do cost and ranking scale with memory size? (§5.3)
- **RQ4**: Does cross-tool sharing work? (§5.4)

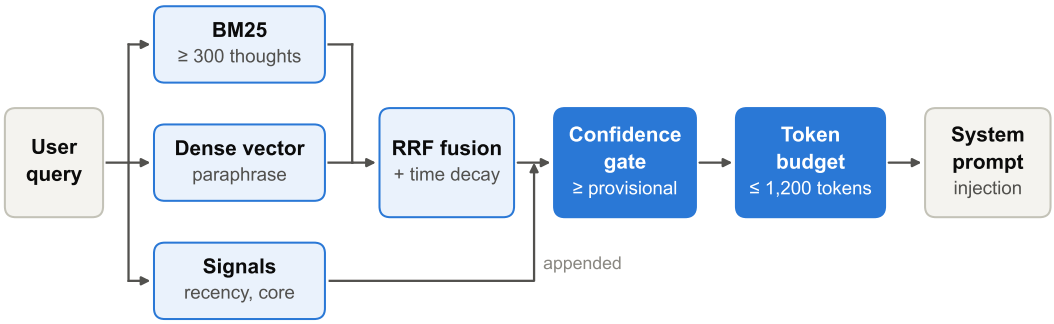


Figure 2: Retrieval and injection pipeline. Dense and BM25 rankings fuse via RRF (BM25 from 300 thoughts); recency and core-fact thoughts are appended; a confidence gate removes non-injectable thoughts; a token budget caps injected context.

Models. Gemini Flash-Lite (cloud; `gemini-flash-lite-latest`, queried 30 June 2026), gemma3:4b and qwen2.5:3b (local via Ollama). Answers used a 150-token limit and default sampling, one per question and condition; the Gemini alias and Ollama tags are mutable, and local model digests were not recorded.

A/B design. Each of 30 synthetic English profiles consists of 5 build statements (identity, preferences, positions, decisions) followed by 5 test questions with a known expected keyword. Each statement is made once and firmly ("I prefer…"), which is enough to reach provisional (§3.2). Each question is answered *with memory* (through the Noesis proxy) and *without* (direct to the provider); all models get the identical first 110 questions (22 profiles). An answer counts as a hit if it contains the expected keyword, and I report the keyword-hit rate; paired outcomes are tested with McNemar's χ^2 [17].

Retrieval-layer experiments. Ablation, scale, and cross-tool experiments call the real pipeline's Python API without LLM calls, measuring Recall@5, MRR, the recall and size of the injected context, tentative leakage, and injected tokens; the cross-tool experiment simulates tools by labeling each statement's source.

Integration routes. Only the proxy, which injects context and stores each exchange automatically, is evaluated end to end (§5.1). MCP tools are model-controlled [19]: nothing is stored or injected unless the model calls `remember` or `recall`, so timely use is not ensured, and the MCP server has unit tests only. The browser extension only captures turns and is not evaluated.

Reproducibility. A released script (`experiments/verify_paper.py`) recomputes every experimental number in this paper from the raw result files and reads the lifecycle constants from the source. The ablation and scale experiments were re-run in October 2026 with the June software (Python 3.12.11, torch 2.2.2, transformers 4.44.2, sentence-transformers 3.0.1, all-MiniLM-L6-v2) and reproduced the June numbers exactly before the metric corrections (§9) and the top-5 baseline were added. The script audits arithmetic, not answer scoring, and the saved records are incomplete: the local-model A/B runs kept the first 150 characters of each answer and the end-to-end run the first 100 (hits were scored on full answers); the Gemini A/B records were recovered from the run's console log, which printed only hit marks; and no run kept prompts or injected contexts. Answers

therefore cannot be re-scored from the released files. Code, data, and paper sources are tagged `v1.0-preprint`.

5 Results

5.1 RQ1: Memory injection raises the keyword-hit rate across model tiers

Table 3: A/B keyword-hit rates on identical n=110 questions. All $p < 0.001$ (McNemar, df=1).

Model	n	**With**	**Without**	Δpp	χ^2
Gemini Flash-Lite	110	85.5%	41.8%	**+43.6**	42.48
gemma3:4b	110	82.7%	34.5%	**+48.2**	51.02
qwen2.5:3b	110	62.7%	37.3%	**+25.5**	16.57

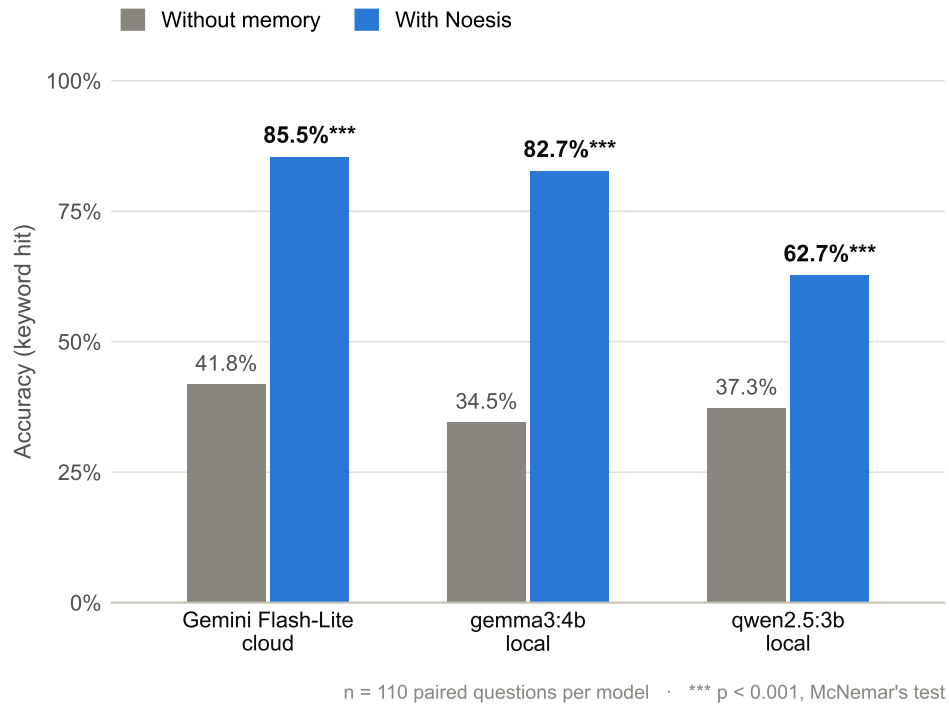


Figure 3: A/B keyword-hit rate across models (n=110 each). All improvements significant at $p < 0.001$.

Table 3 and Figure 3 show the headline result. First, *injection helps everywhere*: all three models improve significantly, and memory *hurt* in only 10 of 330 paired answers (3.0%). Because a profile's five questions share one memory store, I also resampled whole profiles (20,000 draws over 22 profiles): the 95% intervals for the gains are +34.5 to +51.8pp (Gemini Flash-Lite), +37.3 to +59.1pp (gemma3:4b), and +13.6 to +36.4pp (qwen2.5:3b), all above zero. Second, *memory levels the field between local and cloud*: with injection, gemma3:4b reaches 82.7%, close to Gemini Flash-Lite's 85.5%, from a lower baseline (34.5% vs. 41.8%). Third, *the smallest model gains least*:

qwen2.5:3b reaches only 62.7%; a model-capability ceiling is one possible reason, but the saved runs do not record the injected context, so it remains a hypothesis.

The hit rate alone does not separate retrieval strategies at this scale. In a separate end-to-end run (50 questions, 10 profiles, Gemini Flash-Lite), Noesis, injecting every stored memory, and injecting the five most recent scored 41, 40, and 41 of 50, against 22 with no memory; with only 3–9 memories per profile, all three inject nearly the same context. The A/B result therefore shows that personal context helps on these tasks, not that the lifecycle beats simpler ways of supplying the same memories.

5.2 RQ2: Semantic retrieval drives recall; gating keeps tentative thoughts out

Table 4: Ablation and baselines (50 queries over 28 memories, 5 tentative; retrieval-layer, 2,000-token budget). A retrieved memory counts only if it is the ground-truth memory. Recall@5 and MRR use the first five memories in injection order (or all, if fewer are returned); context recall, items, and leak cover everything injected, and leak counts tentative memories among those slots.

Config	Recall@5	MRR	Context recall	Items	Leak
Noesis (full)	**99%**	**1.000**	**100%**	**10.9**	**0/546**
w/o core-fact signal	99%	1.000	100%	9.7	0/487
w/o recency signal	99%	1.000	100%	9.7	0/484
w/o confidence gating	99%	1.000	100%	11.5	**73/573**
w/o semantic retrieval	24%	0.105	24%	5.0	0/250
Top-5 RAG	99%	1.000	99%	5.0	19/250
Recent-window	17%	0.084	17%	5.0	50/250
Random	12%	0.063	12%	5.0	51/250

Each Noesis variant runs the real context builder with one component switched off; the baselines draw from every stored memory, tentative ones included. Status and confidence are assigned when the test store is built, so the suite tests whether retrieval respects those labels, not how the lifecycle derives them. At 28 memories the semantic channel is dense retrieval alone, and stale-memory leakage is measured only by the supersession test (Table 2).

Removing semantic retrieval collapses recall (99%→24%; Table 4). Removing the confidence gate leaves recall unchanged but lets tentative thoughts into 73 of 573 injected slots, where the full system injects none. Ungated top-5 retrieval ranks as well as Noesis (Recall@5 99%, MRR 1.000), as it does at scale (§5.3), but lets a tentative thought into 19 of its 250 slots. A separate poisoning test agrees: with 5 settled facts and 8 hedged statements contradicting them, ungated top-5 retrieval returned a hedged statement in 13 of 25 slots across 5 queries, and Noesis returned none. Both tests use hedged statements; a firmly stated false one would pass the gate (§9). Removing the core-fact or recency signal does not change recall: each adds about one memory per query (10.9 vs. 9.7 items), and because the builder appends them after the eight semantic results, they never reach the first five slots. Whether they help on queries that semantic retrieval misses needs a dedicated query set.

5.3 RQ3: Retrieval keeps cost flat as memory grows

Figure 4 sweeps memory size from 10 to 500 with 10 known targets embedded among fillers. Noesis's injected cost is flat (144→207 tokens), well under the 1,200-token budget, and MRR holds at 0.90:

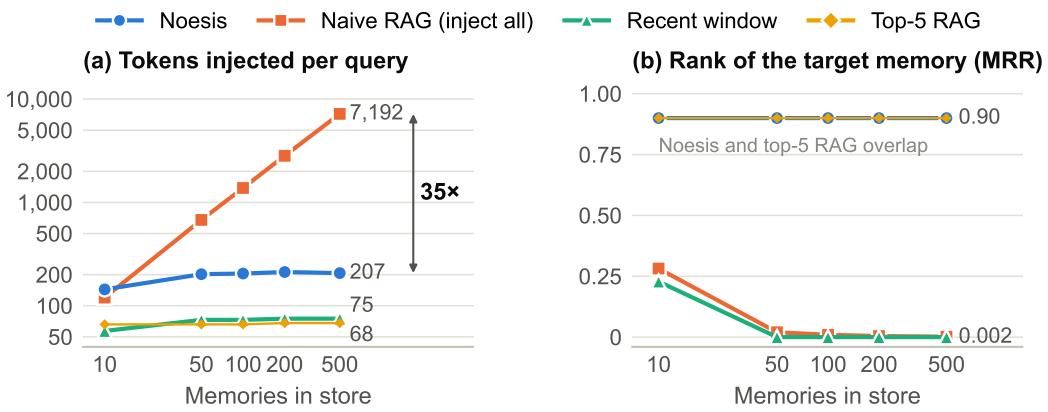


Figure 4: Scale behavior from 10 to 500 stored memories. (a) Noesis's injected tokens stay flat (∼207) while naive RAG (inject all) grows to 7,192 (35 times); top-5 RAG stays at 66–68. (b) Noesis and top-5 RAG hold MRR at 0.90; naive RAG collapses to 0.002 as targets are buried; recent-window loses recall entirely past n=10.

9 of 10 targets sit at rank 1 and the tenth is never retrieved. Naive RAG keeps recall only by injecting everything: 7,192 tokens at n=500, with the target buried near rank 500 (MRR 0.002). Recent-window recall collapses to 0% beyond n=10. A standard top-5 RAG (the query's five nearest memories) does as well: the same 9 of 10 targets at rank 1, with 66–68 tokens per query. Flat cost and good ranking come from retrieval itself, not from anything specific to Noesis; what Noesis adds is the gate (§5.2). Recall alone is therefore misleading; rank and token cost must be reported together.

5.4 RQ4: Cross-tool sharing vs. structural isolation

Of 40 scenarios, 27 state a stance in one tool and ask about it from another; in 7 the asking tool was one of two that stated it; in 6 a third tool asks after two others did (Figure 5). On the 27 strictly cross-tool queries Noesis retrieves 37% (10/27) and the isolated baseline 0%, by construction, since the memory is not in its partition. Where the asking tool did see the stance (7 scenarios), isolated memory retrieves 3 and Noesis 6; over all 40, Noesis retrieves 50% and the isolated baseline 7.5%. The comparison shows the value of access, not an advantage over another shared-memory system given the same records. Noesis's misses trace to retrieval precision (the memory exists but ranks below top-k), not to sharing.

6 The Star Map Visualization

The star map shows the stored thoughts as a galaxy (Figure 6), placed by a short force simulation: weak springs pull each star toward its creation time (horizontal) and confidence (vertical), semantic attraction pulls it toward its most similar neighbors (cosine ≥ 0.45), and repulsion separates topics. Clusters therefore form from embedding similarity while time and confidence stay readable, and hovering a star shows its text and confidence. The store shown is the local development store, filled by experiment and demo runs: of its 601 thoughts, 355 sit in the catch-all "general" topic and 18 have settled, which shows how coarse topic assignment still is. The viewer opens the store read-only, and its edges never affect retrieval, so the map says nothing about whether graph structure

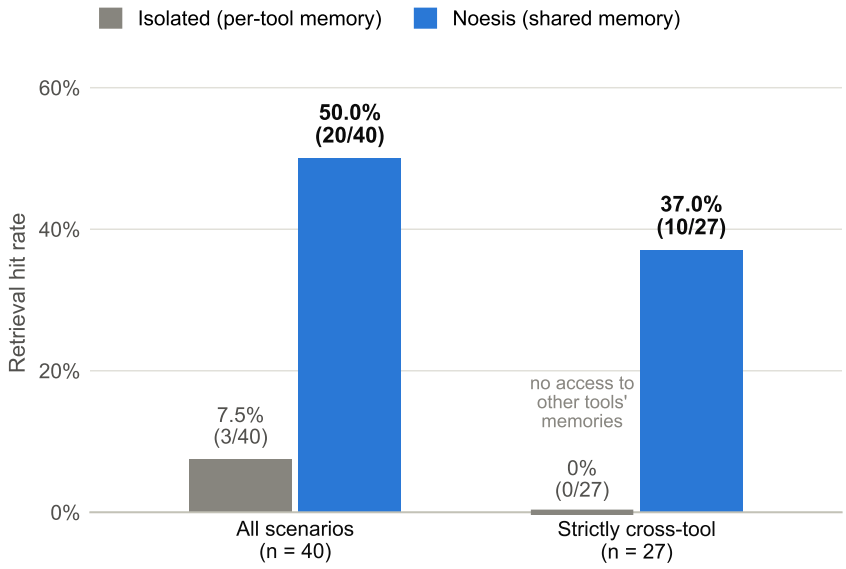


Figure 5: Cross-tool retrieval. Isolated per-tool memory scores 0% on strictly cross-tool queries by construction.

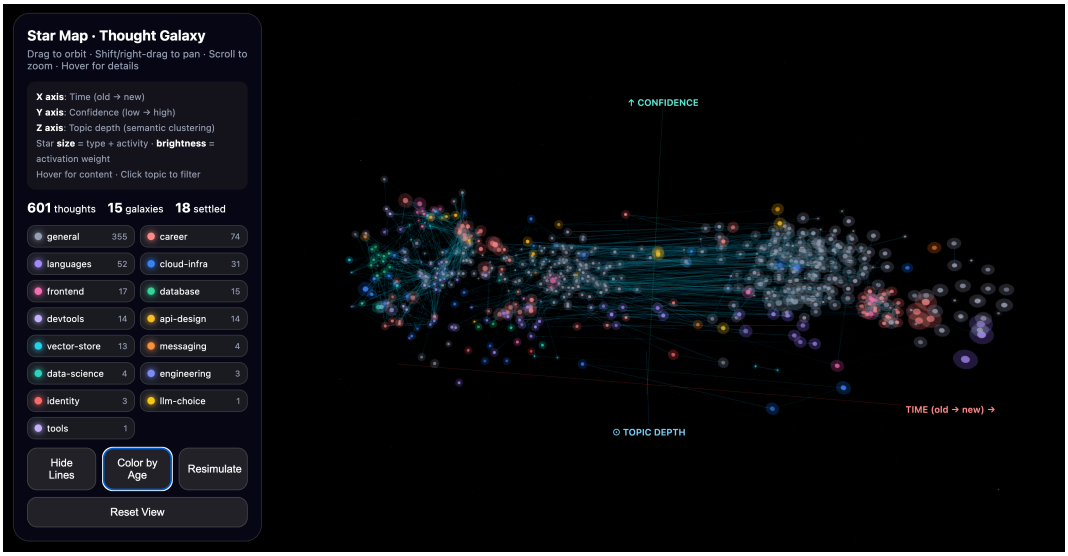


Figure 6: The star map. Semantic neighbors cluster; time runs from older (left) to newer (right), height shows confidence, color shows topic, and size and brightness show type and activation weight.

improves answers.

7 Discussion

When memory helps most. Gains concentrate on questions unanswerable without personal context (job, stack, stances); on mainstream questions both conditions often answer correctly. Other factors push the other way (§9), so these gains should not be read as an upper or lower bound.

Memory can close the local–cloud gap. gemma3:4b gains most (+48.2pp; +139% over its baseline) and approaches Gemini Flash-Lite with memory, while qwen2.5:3b gains least (+25.5pp; §5.1), so the benefit is not monotonic in model size. It is largest for small local models that can use the injected context, which matters for private and offline use.

Retrieval precision is the current bottleneck. The cross-tool misses and several A/B misses are ranking failures: the memory is stored but ranks below the cut, so better ranking (rerankers, query expansion) is the highest-leverage next step. The supersession fix's residual 7% is different: in both leaks the detector never fired, because its lexicon lacked the replacement verb in one case and the domain (Node.js) in the other.

Noesis inside an agent harness. An execution harness manages a task's control flow, tools, state, and recovery [1], and useful scaffolding changes as models improve [2]; Noesis would serve inside one as memory, not orchestration, though this is neither implemented nor evaluated here. The harness would insert approved memories for the current user, project, and scope as a bounded, sourced block and send back candidate memories with their sources; Noesis could fill LangGraph's cross-thread store, not its thread checkpoints [12]. Integration also needs corrections, deletion, audit, and source lineage, and three gaps stand out: repetition counts similar records whatever their source, and the proxy stores each request's full history, so replays and assistant paraphrases can inflate confidence (unmeasured); new memories become injectable only after asynchronous scoring; and injected memories are neither marked as data nor deduplicated against the harness's summaries.

Graph structure. The evaluated system is not a graph-based reasoner. Supersession links are implemented and evaluated (Table 2); topic clusters and wiki links are not evaluated separately; and the one-hop expansion over vault links cannot add a memory (it writes 8-character identifiers but looks up 12-character ones). Every reported result comes from flat retrieval.

Negative results. I report that the supersession check initially never ran, so stale stances leaked 93% of the time (§3.3); that core-fact and recency signals added no recall in my ablation; that top-5 RAG matches Noesis's ranking at lower cost; that the 3B model gains least; and that the graph expansion never adds a memory. I consider these findings part of the contribution.

8 Future Work

Thought branching. When a new judgment deviates from history by more than a threshold ($\delta = 1 - \cos > 0.45$), Noesis should branch via `evolved_from` rather than supersede, preserving a stance's trajectory and enabling metrics such as stabilization depth and branching rate.

Structured injection. Replacing flat context lines with a structured `user_thought_graph` may help small models use injected memory. Typed, timed claims with support, contradiction, derivation, and supersession relations would let "PostgreSQL for analytics" and "MySQL for a legacy app" coexist, as in Zep [22]; derived claims are not independent evidence, so graph centrality should not raise confidence, and the deciding test crosses flat and graph-backed stores with and without the same gate. A graph neural network [3] would also need held-out users, domains, and time periods.

Stronger evaluation. LLM-as-judge scoring to complement keyword hits; distractor questions; a second cloud model; longitudinal data beyond synthetic profiles; and a harness-level comparison of native harness memory, a simple shared store, and Noesis, with model, tasks, tools, and budget fixed. Questions about alternatives the user was considering need tentative thoughts, so excluding them should be compared with retrieving them with their uncertainty and source, adapting the risk–coverage view of selective prediction [8]. Counting repetition only across distinct user statements would close the first gap in §7.

9 Limitations

Keyword-hit scoring checks whether an answer contains the expected keyword, not whether it is right: "you do not prefer Python" contains the same keyword as "you prefer Python", and a correct answer worded differently or in another language counts as a miss. Personas are synthetic, build/test alignment may overstate real gains, and real conversations are noisier. The cloud A/B uses one provider and n=110 rather than the planned 150: rate limits stopped the 30 June run after 111 questions, and I did not add the 35 answered in a later session because the `gemini-flash-lite-latest` alias may by then have pointed to another model. Retrieval-layer experiments measure what reaches the prompt, not answer quality. The supersession detector relies on replacement verbs and a domain keyword map, and its 93% → 7% result is in-sample: it was developed on the same 30 scenarios, and held-out paraphrases, unseen domains, and legitimate simultaneous preferences (which a domain match could wrongly retire) are needed. On the scale benchmark top-5 RAG ties Noesis on ranking. The ablation and poisoning leak counts were corrected in October 2026 to stop counting settled memories that merely shared a word with a tentative one (for example, "Kafka"). After an external review, I changed the ablation in five ways: it scores exact memory identity instead of shared phrases such as "I prefer"; it drops "Precision@5", which measured non-tentative results rather than relevant ones; it runs every variant through the same context builder, so each differs from the full system in one component only; it applies the memories' ages, which the test store had dropped; and it lets the recent-window and random baselines return tentative memories. This lowered the weak baselines (random 45% → 12%, w/o semantic 49% → 24% Recall@5); Noesis moved from 100% to 99%. No result has been independently replicated.

Noesis also adds an attack surface: the local proxy sees every request, including provider API keys and full conversations; optional cloud extraction sends conversation text to a third party; and because one firmly stated sentence reaches provisional (§3.2), a planted or mistaken statement can enter future prompts. The gate filters hedged statements but is no defense against deliberate poisoning.

10 Conclusion

Injecting a user's own memories raised all three models' keyword-hit rates on personal questions, though whether Noesis's lifecycle beats simpler ways of supplying them is not yet shown. Its gate

kept tentative and hedged statements out of the prompt in constructed tests where plain retrieval let them in. Retrieval keeps cost flat (35 times cheaper than injecting everything at n=500; top-5 RAG is cheaper still), and a shared per-user store gives every tool the same records. The result is a local-first layer that brings a 4B local model's keyword-hit rate close to a small cloud model's (82.7% vs. 85.5%; the gap is not significant, though that does not make them equal) and keeps the user's reasoning in the user's own vault.

Acknowledgments

AI coding assistants were used in this work. ZCode (Z.ai, GLM models) assisted with implementing the experiment scripts and drafting the original experiment reports. Claude (Anthropic) was used to review the manuscript, revise its text, redesign the figures, write the verification script (`experiments/verify_paper.py`), and re-run the retrieval-layer experiments in October 2026. All reported numbers are checked against the raw results by that script, and the author takes full responsibility for the paper's content.

References

[1] Anthropic. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025. Engineering blog, November 26, 2025.

[2] Anthropic. Harness design for long-running application development. https://www.anthropic.com/engineering/harness-design-long-running-apps, 2026. Engineering blog, March 24, 2026.

[3] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

[4] Prateek Chhikara, Dev Khant, Sparsh Aryan, et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.

[5] Shen Dong, Shaochen Xu, Pengfei He, et al. Memory injection attacks on LLM agents via query-only interaction. *arXiv preprint arXiv:2503.03704*, 2025.

[6] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[7] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991.

[8] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[10] Yuyang Hu, Shichun Liu, Yanwei Yue, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025.

[11] Shuo Ji, Yibo Li, and Bryan Hooi. Memory is reconstructed, not retrieved: Graph memory for LLM agents. *arXiv preprint arXiv:2606.06036*, 2026. ICML 2026.

[12] LangChain. Persistence. https://docs.langchain.com/oss/python/langgraph/persistence, 2026. LangGraph documentation, accessed October 17, 2026.

[13] Christopher Latimer, Nicolò Boschi, Andrew Neeser, et al. Hindsight: Structured agent memory that retains, recalls, and reflects. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 275–285, 2026. doi: 10.18653/v1/2026.acl-demo.27.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[15] Zehao Lin, Xixuan Hao, Renyu Fu, et al. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. *arXiv preprint arXiv:2604.16548*, 2026. EMNLP 2026.

[16] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[17] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[18] Mem0. Introducing OpenMemory MCP. https://

mem0.ai/blog/introducing-openmemory-mcp, 2025. Vendor announcement, Mem0 blog, May 13, 2025.

[19] Model Context Protocol. Model context protocol specification, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/architecture, 2026. Accessed October 17, 2026.

[20] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[21] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[22] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[23] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

[24] Di Wu, Zixiang Ji, Asmi Kawatkar, et al. LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*, 2026. Work in progress.

[25] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[26] Chang Yang, Chuang Zhou, Yilin Xiao, et al. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*, 2026.

[27] Yi Yu, Liuyi Yao, Yuexiang Xie, et al. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026. ACL 2026.

[28] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.